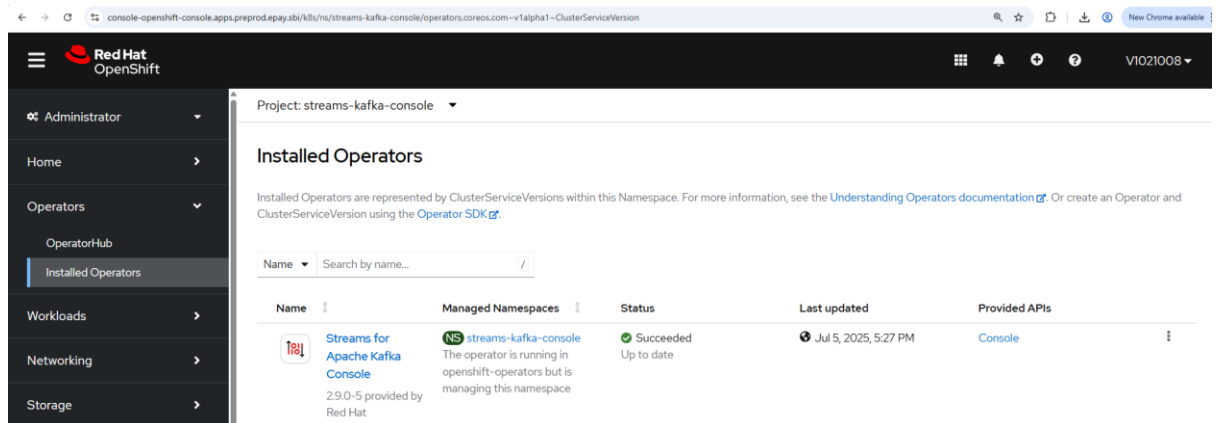
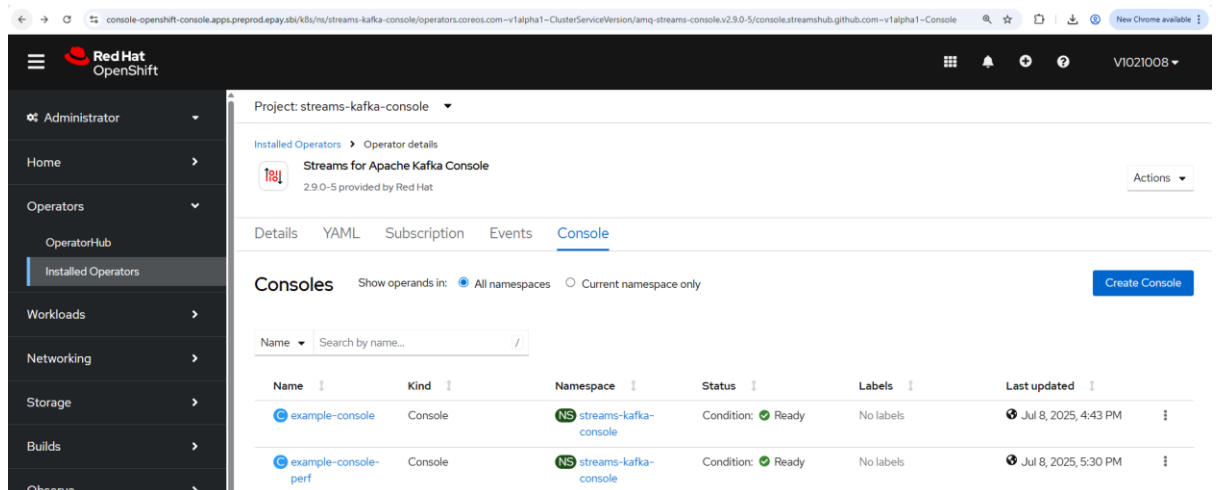


Steps to create and configure Kafka Console in Pre-prod Env:

1. Make sure Kafka console operator should be already installed in OpenShift cluster.



2. Now click on “Apache Kafka Console” followed by “Console” and from here we can click on “Create Console”.



Now edit the below Sample yaml file and saved it.

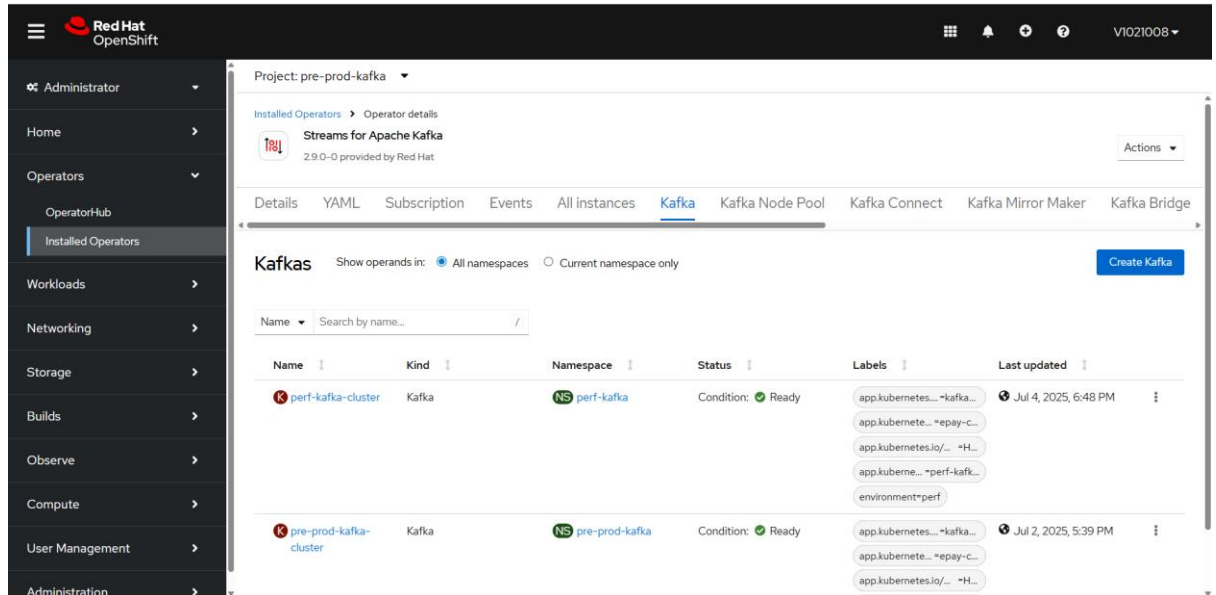
```
kind: Console
apiVersion: console.streamshub.github.com/v1alpha1
metadata:
  name: example-console
  namespace: streams-kafka-console
spec:
  hostname: my-console.apps.preprod.epay.sbi
  kafkaClusters:
    - adminProperties:
        values: []
        valuesFrom: []
      consumerProperties:
        values: []
        valuesFrom: []
      credentials:
        kafkaUser:
          name: my-user
      listener: plain
      name: pre-prod-kafka-cluster
      namespace: pre-prod-kafka
      producerProperties:
        values: []
```

```

valuesFrom: []
properties:
  values: []
  valuesFrom: []

```

3. After create go to Apache Stream Operator and Click on “Pre-prod-Kafka-cluster” and verify the listener value from yaml file.



```

165 listeners:
166   - authentication:
167     type: scram-sha-512
168     name: plain
169     port: 9092
170     tls: false
171     type: internal
172   - name: tls
173     port: 9093
174     tls: true
175     type: internal
176   - name: external
177     port: 9094
178     tls: true
179     type: route
180 metadataVersion: 3.9-IV0
181 metricsConfig:
182   type: jmxPrometheusExporter
183   valueFrom:
184     configMapKeyRef:
185       key: kafka-metrics-config.yml
186       name: kafka-metrics
187 version: 3.9.0

```

4. This Configmap yaml metric file should be present in Pre-prod-kafka or respective namespace.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: kafka-metrics
  labels:
    app: strimzi
data:
  kafka-metrics-config.yml: |
    # See https://github.com/prometheus/jmx_exporter for more info about JMX Prometheus Exporter
metrics
  lowercaseOutputName: true
  rules:
    # Special cases and very specific rules
    - pattern: kafka.server<type=(.+), name=(.+), clientId=(.+), topic=(.+), partition=(.*)><>Value
      name: kafka_server_$1_$2
      type: GAUGE
      labels:
        clientId: "$3"
        topic: "$4"
        partition: "$5"
    - pattern: kafka.server<type=(.+), name=(.+), clientId=(.+), brokerHost=(.+),
brokerPort=(.+)><>Value
      name: kafka_server_$1_$2
      type: GAUGE
      labels:
        clientId: "$3"
        broker: "$4:$5"
    - pattern: kafka.server<type=(.+), cipher=(.+), protocol=(.+), listener=(.+),
networkProcessor=(.+)><>connections
      name: kafka_server_$1_connections_tls_info
      type: GAUGE
      labels:
        cipher: "$2"
        protocol: "$3"
        listener: "$4"
        networkProcessor: "$5"
    - pattern: kafka.server<type=(.+), clientSoftwareName=(.+), clientSoftwareVersion=(.+),
listener=(.+), networkProcessor=(.+)><>connections
      name: kafka_server_$1_connections_software
      type: GAUGE
      labels:
        clientSoftwareName: "$2"
        clientSoftwareVersion: "$3"
        listener: "$4"
        networkProcessor: "$5"
    - pattern: "kafka.server<type=(.+), listener=(.+), networkProcessor=(.+)><>(.+-total):"
      name: kafka_server_$1_$4
      type: COUNTER
      labels:
        listener: "$2"
        networkProcessor: "$3"
    - pattern: "kafka.server<type=(.+), listener=(.+), networkProcessor=(.+)><>(.+):"
      name: kafka_server_$1_$4
      type: GAUGE
      labels:
```

```

    listener: "$2"
    networkProcessor: "$3"
- pattern: kafka.server<type=(.+), listener=(.+), networkProcessor=(.)><>(.+-total)
  name: kafka_server_$1_$4
  type: COUNTER
  labels:
    listener: "$2"
    networkProcessor: "$3"
- pattern: kafka.server<type=(.+), listener=(.+), networkProcessor=(.)><>(.+)
  name: kafka_server_$1_$4
  type: GAUGE
  labels:
    listener: "$2"
    networkProcessor: "$3"
# Some percent metrics use MeanRate attribute
# Ex) kafka.server<type=(KafkaRequestHandlerPool),
name=(RequestHandlerAvgIdlePercent)><>MeanRate
- pattern: kafka.(\\w+)<type=(.+), name=(.)Percent\\w*><>MeanRate
  name: kafka_$1_$2_$3_percent
  type: GAUGE
# Generic gauges for percents
- pattern: kafka.(\\w+)<type=(.+), name=(.)Percent\\w*><>Value
  name: kafka_$1_$2_$3_percent
  type: GAUGE
- pattern: kafka.(\\w+)<type=(.+), name=(.)Percent\\w*, (.+)=(.+)><>Value
  name: kafka_$1_$2_$3_percent
  type: GAUGE
  labels:
    "$4": "$5"
# Generic per-second counters with 0-2 key/value pairs
- pattern: kafka.(\\w+)<type=(.+), name=(.)PerSec\\w*, (.+)=(.+), (.+)=(.+)><>Count
  name: kafka_$1_$2_$3_total
  type: COUNTER
  labels:
    "$4": "$5"
    "$6": "$7"
- pattern: kafka.(\\w+)<type=(.+), name=(.)PerSec\\w*, (.+)=(.+)><>Count
  name: kafka_$1_$2_$3_total
  type: COUNTER
  labels:
    "$4": "$5"
- pattern: kafka.(\\w+)<type=(.+), name=(.)PerSec\\w*><>Count
  name: kafka_$1_$2_$3_total
  type: COUNTER
# Generic gauges with 0-2 key/value pairs
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(.+), (.+)=(.+)><>Value
  name: kafka_$1_$2_$3
  type: GAUGE
  labels:
    "$4": "$5"
    "$6": "$7"
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(.+)><>Value
  name: kafka_$1_$2_$3
  type: GAUGE
  labels:
    "$4": "$5"
- pattern: kafka.(\\w+)<type=(.+), name=(.)><>Value

```

```

name: kafka_$1_$2_$3
type: GAUGE
# Emulate Prometheus 'Summary' metrics for the exported 'Histogram's.
# Note that these are missing the '_sum' metric!
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(.+), (.+)=(.+)><>Count
  name: kafka_$1_$2_$3_count
  type: COUNTER
  labels:
    "$4": "$5"
    "$6": "$7"
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(*), (.+)=(.+)><>(\\d+)thPercentile
  name: kafka_$1_$2_$3
  type: GAUGE
  labels:
    "$4": "$5"
    "$6": "$7"
    quantile: "0.$8"
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(.+)><>Count
  name: kafka_$1_$2_$3_count
  type: COUNTER
  labels:
    "$4": "$5"
- pattern: kafka.(\\w+)<type=(.+), name=(.+), (.+)=(*)><>(\\d+)thPercentile
  name: kafka_$1_$2_$3
  type: GAUGE
  labels:
    "$4": "$5"
    quantile: "0.$6"
- pattern: kafka.(\\w+)<type=(.+), name=(.+)><>Count
  name: kafka_$1_$2_$3_count
  type: COUNTER
- pattern: kafka.(\\w+)<type=(.+), name=(.+)><>(\\d+)thPercentile
  name: kafka_$1_$2_$3
  type: GAUGE
  labels:
    quantile: "0.$4"
# KRaft overall related metrics
# distinguish between always increasing COUNTER (total and max) and variable GAUGE (all others)
metrics
- pattern: "kafka.server<type=raft-metrics><>(.+-total|.+-max):"
  name: kafka_server_raftmetrics_$1
  type: COUNTER
- pattern: "kafka.server<type=raft-metrics><>(current-state): (.+)"
  name: kafka_server_raftmetrics_$1
  value: 1
  type: UNTYPED
  labels:
    $1: "$2"
- pattern: "kafka.server<type=raft-metrics><>(.+):"
  name: kafka_server_raftmetrics_$1
  type: GAUGE
# KRaft "low level" channels related metrics
# distinguish between always increasing COUNTER (total and max) and variable GAUGE (all others)
metrics
- pattern: "kafka.server<type=raft-channel-metrics><>(.+-total|.+-max):"
  name: kafka_server_raftchannelmetrics_$1
  type: COUNTER

```

- pattern: "kafka.server<type=raft-channel-metrics><>(.+):"
name: kafka_server_raftchannelmetrics_\$1
type: GAUGE
- # Broker metrics related to fetching metadata topic records in KRaft mode
- pattern: "kafka.server<type=broker-metadata-metrics><>(.+):"
name: kafka_server_brokermetadatametrics_\$1
type: GAUGE

5. And Below yaml file need to apply as well in the pre-prod-kafka namespace.

```

apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: cluster-operator-metrics
  labels:
    app: strimzi
spec:
  selector:
    matchLabels:
      strimzi.io/kind: cluster-operator
  namespaceSelector:
    matchNames:
      - Pre-prod-kafka
  podMetricsEndpoints:
    - path: /metrics
      port: http
---
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: entity-operator-metrics
  labels:
    app: strimzi
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: entity-operator
  namespaceSelector:
    matchNames:
      - Pre-prod-kafka
  podMetricsEndpoints:
    - path: /metrics
      port: healthcheck
---
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: bridge-metrics
  labels:
    app: strimzi
spec:
  selector:
    matchLabels:
      strimzi.io/kind: KafkaBridge
  namespaceSelector:
    matchNames:
      - Pre-prod-kafka
  podMetricsEndpoints:
    - path: /metrics
      port: rest-api
---

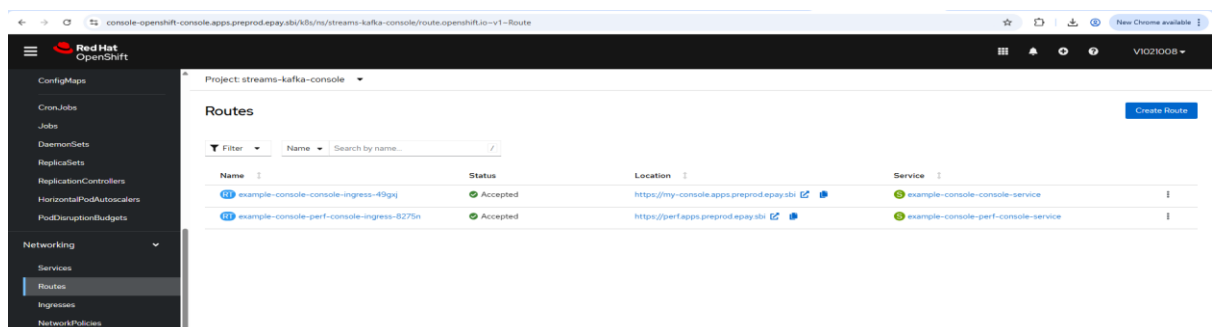
```

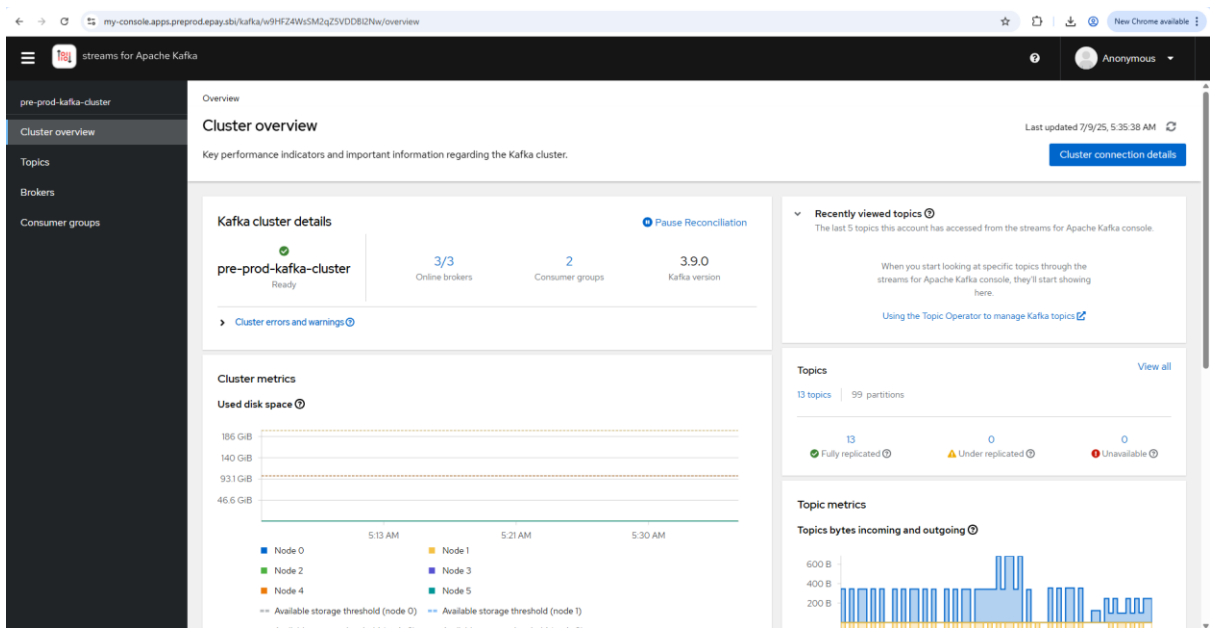
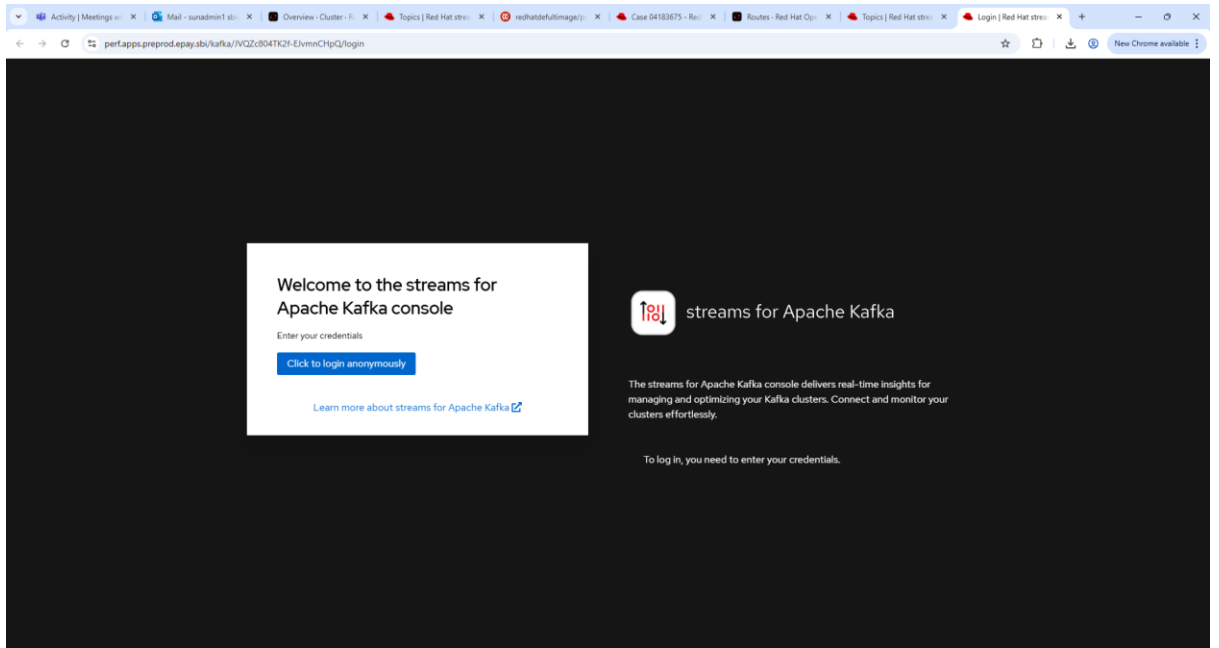
```

apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: kafka-resources-metrics
  labels:
    app: strimzi
spec:
  selector:
    matchExpressions:
      - key: "strimzi.io/kind"
        operator: In
        values: ["Kafka", "KafkaConnect", "KafkaMirrorMaker", "KafkaMirrorMaker2"]
  namespaceSelector:
    matchNames:
      - Pre-prod-kafka
  podMetricsEndpoints:
    - path: /metrics
      port: tcp-prometheus
  relabelings:
    - separator: ;
      regex: __meta_kubernetes_pod_label_(strimzi_io_.+)
      replacement: $1
      action: labelmap
    - sourceLabels: [__meta_kubernetes_namespace]
      separator: ;
      regex: (.*)
      targetLabel: namespace
      replacement: $1
      action: replace
    - sourceLabels: [__meta_kubernetes_pod_name]
      separator: ;
      regex: (.*)
      targetLabel: kubernetes_pod_name
      replacement: $1
      action: replace
    - sourceLabels: [__meta_kubernetes_pod_node_name]
      separator: ;
      regex: (.*)
      targetLabel: node_name
      replacement: $1
      action: replace
    - sourceLabels: [__meta_kubernetes_pod_host_ip]
      separator: ;
      regex: (.*)
      targetLabel: node_ip
      replacement: $1
      action: replace

```

- Once above steps completed, we can select “streams-kafka-console” Namespace from OCP Console and click on Location to see the Kafka Console.





=====EOD=====